

Part 1 — Fundamentals of Information Technology

This part covers the basic building blocks of Information Technology as tested in CPGET: how data becomes information, how computers acquire and represent numbers and text, the CPU, networks, I/O devices, software categories, common software problems, and sound programming principles.

1.1 Data and Information

Concept

Data is raw, unprocessed facts — numbers, characters, or signals — that by themselves carry no context (e.g., 72, 'Ravi', 08). Information is data that has been processed, organized, or structured so it becomes meaningful and useful for decision-making (e.g., 'Ravi scored 72 marks on 08-06-2026'). The transformation of data into information happens through processing: sorting, classifying, summarizing, and calculating. Key properties of good information are accuracy, timeliness, completeness, relevance, and reliability. In the data-information-knowledge-wisdom (DIKW) hierarchy, data sits at the base, information is data with context, knowledge is information with understanding of patterns, and wisdom is knowledge applied with judgement. CPGET frequently tests the distinction between data and information, examples of each, and characteristics of quality information.

Practice Questions

Q1. Which of the following is an example of 'information' rather than raw 'data'?

Solution: 'The average monthly rainfall in Hyderabad in June is 120 mm' is information because raw daily rainfall readings have been processed (averaged) and given context. A single reading like '15 mm' alone is data.

Q2. List three properties that make information 'good quality' for decision-making.

Solution: Accuracy (free from errors), Timeliness (available when needed), and Relevance (applicable to the decision at hand). Completeness and reliability are also acceptable answers.

Q3. Explain with one example how the same data can produce different information for different users.

Solution: Sales figures (data) of a shop can be processed into 'total monthly revenue' for an owner, or into 'best-selling product' for a marketing team — same raw data, different processing goals, different information.

Previous Year Questions

1. Data can best be described as:\n(A) Processed facts \n(B) Raw, unorganized facts \n(C) Knowledge derived from experience \n(D) A decision-making tool

Answer: (B)

2. Which term refers to data that has been given context and meaning?\n(A) Database \n(B) Information \n(C) Algorithm \n(D) Protocol

Answer: (B)

3. In the DIKW hierarchy, which level comes immediately above 'Data'?\n(A) Wisdom \n(B) Knowledge \n(C) Information \n(D) Intelligence

Answer: (C)

1.2 Acquisition of Numbers and Textual Data

Concept

Computers internally represent everything in binary (0/1). Numeric data acquisition involves converting decimal numbers into binary using number systems: binary (base 2), octal (base 8), decimal (base 10), and hexadecimal (base 16). Conversions between these systems (decimal↔binary↔octal↔hex) are core exam skills. Textual data is acquired using character-encoding schemes that map each character to a numeric code: ASCII (7-bit, 128 characters, covers English letters/digits/symbols), Extended ASCII (8-bit, 256 characters), and Unicode (16-bit or more, supports virtually all world scripts including Telugu, Hindi, etc., commonly implemented as UTF-8/UTF-16). Unicode was developed to overcome ASCII's limitation of representing only English characters. Input acquisition methods include keyboards, scanners (OCR for text), and sensors, each converting real-world signals into digital data.

Practice Questions

Q1. Convert the decimal number 45 into binary.

Solution: $45 = 32 + 8 + 4 + 1 = 101101$. (Divide by 2 repeatedly: $45 \rightarrow 22$ r1, $22 \rightarrow 11$ r0, $11 \rightarrow 5$ r1, $5 \rightarrow 2$ r1, $2 \rightarrow 1$ r0, $1 \rightarrow 0$ r1; reading remainders bottom-up gives 101101).

Q2. Why was Unicode introduced when ASCII already existed?

Solution: ASCII uses only 7–8 bits and can represent just 128–256 characters, sufficient only for English and a few symbols. Unicode uses 16+ bits, allowing over 65,000+ characters so that scripts like Telugu, Hindi, Chinese, and Arabic can also be digitally represented.

Q3. Convert the hexadecimal number 2F to decimal.

Solution: $2F = 2 \times 16 + 15 \times 1 = 32 + 15 = 47$.

Previous Year Questions

1. How many bits does standard ASCII use to represent one character? \n(A) 4 \n(B) 7 \n(C) 16 \n(D) 32

Answer: (B)

2. The binary equivalent of decimal 10 is: \n(A) 1010 \n(B) 1100 \n(C) 1001 \n(D) 1110

Answer: (A)

3. Which encoding standard is designed to support multiple world languages including Indian scripts? \n(A) ASCII \n(B) EBCDIC \n(C) Unicode \n(D) BCD

Answer: (C)

4. The number system with base 16 is called: \n(A) Binary \n(B) Octal \n(C) Decimal \n(D) Hexadecimal

Answer: (D)

1.3 Central Processing Unit (CPU)

Concept

The CPU is the 'brain' of the computer and has three main components: the Control Unit (CU), which directs and coordinates operations by fetching, decoding, and directing execution of instructions; the Arithmetic Logic Unit (ALU), which performs arithmetic (add, subtract) and logical (AND, OR, comparison) operations; and Registers, small high-speed storage locations inside the CPU used to hold data/instructions temporarily during processing (e.g., Accumulator, Program Counter, Instruction Register, Memory Address Register). The CPU executes instructions in a repeating Fetch–Decode–Execute cycle. Clock speed (measured in GHz) indicates how many cycles the CPU performs per second. Cache memory (L1/L2/L3) is fast memory close to the CPU that stores frequently used data to speed up processing. Multi-core CPUs contain multiple processing units to execute several instruction streams in parallel.

Practice Questions

Q1. Name the three components of the CPU and state one function of each.

Solution: 1) Control Unit — directs/coordinates fetch-decode-execute of instructions. 2) ALU — performs arithmetic and logical operations. 3) Registers — temporarily hold data/instructions/addresses during processing.

Q2. What is the purpose of cache memory?

Solution: Cache memory is small, very fast memory located near/inside the CPU that stores frequently or recently used data and instructions, reducing the time the CPU spends waiting for data from slower main memory (RAM).

Q3. Describe the Fetch–Decode–Execute cycle in one line each.

Solution: Fetch: CPU retrieves the next instruction from memory. Decode: Control unit interprets what the instruction means. Execute: ALU/other units carry out the operation specified.

Previous Year Questions

1. Which CPU component performs arithmetic and logical operations? \n(A) Control Unit \n(B) ALU \n(C) Register \n(D) Cache

Answer: (B)

2. The register that holds the address of the next instruction to be fetched is the:\n(A) Accumulator \n(B) Program Counter \n(C) Instruction Register \n(D) Status Register

Answer: (B)

3. The basic operational cycle of a CPU is called the:\n(A) Read-Write cycle \n(B) Fetch-Decode-Execute cycle \n(C) Input-Output cycle \n(D) Compile-Run cycle

Answer: (B)

1.4 Computer Networks

Concept

A computer network connects multiple devices to share resources and data. Classification by geographic size: LAN (Local Area Network, small area like an office/campus), MAN (Metropolitan Area Network, a city), WAN (Wide Area Network, spans countries, e.g., the Internet). Network topologies describe physical/logical layout: Bus, Star, Ring, Mesh, and Hybrid. The OSI model has 7 layers (Physical, Data Link, Network, Transport, Session, Presentation, Application) describing how data moves across a network; TCP/IP is the practical 4-layer protocol suite used on the Internet. Key devices: Hub (broadcasts to all ports), Switch (intelligently forwards to the correct port using MAC address), Router (connects different networks, routes using IP address), and Modem (converts digital $\leftrightarrow$ analog signals). Protocols include HTTP/HTTPS (web), FTP (file transfer), SMTP (email), and TCP/IP (transmission control).

Practice Questions

Q1. Differentiate between a Hub and a Switch.

Solution: A Hub broadcasts incoming data to all connected devices regardless of the intended recipient (works at Physical layer, no intelligence). A Switch examines the destination MAC address and forwards data only to the intended device (works at Data Link layer, more efficient and secure).

Q2. Which network topology connects every device to a central node, and what is one advantage?

Solution: Star topology. Advantage: if one cable/device fails, only that device is affected; the rest of the network keeps working, and it is easy to add new devices.

Q3. What is the main difference between LAN and WAN?

Solution: LAN covers a small geographic area (e.g., single building/campus) with high speed and low cost; WAN spans large geographic areas (cities/countries), typically uses public/leased lines, and has comparatively lower speed and higher cost.

Previous Year Questions

1. The Internet is an example of which type of network?\n(A) LAN \n(B) MAN \n(C) WAN \n(D) PAN

Answer: (C)

2. Which device connects two different networks and routes data using IP addresses?\n(A) Hub \n(B) Switch \n(C) Router \n(D) Repeater

Answer: (C)

3. How many layers does the OSI reference model have?\n(A) 4 \n(B) 5 \n(C) 6 \n(D) 7

Answer: (D)

4. Which topology connects each device to exactly two neighbours forming a closed loop?\n(A) Star \n(B) Bus \n(C) Ring \n(D) Mesh

Answer: (C)

1.5 Input Output Devices

Concept

Input devices convert real-world data/commands into digital form the computer can process: Keyboard, Mouse, Scanner, Joystick, Microphone, Touchscreen, Barcode reader, Biometric sensor. Output devices convert processed digital data back into a human-usable form: Monitor (soft copy), Printer (hard copy — Impact types

like Dot Matrix, and Non-Impact types like Inkjet/Laser), Speaker, Plotter, Projector. Some devices are both input and output, e.g., touchscreens, modems, and network cards. Printers are further classified by technology: Impact printers physically strike an ink ribbon (noisy, cheap, good for carbon copies) while Non-Impact printers (inkjet, laser) spray ink or use toner (quiet, higher quality). Understanding the classification and comparing device types (speed, cost, quality, use-case) is a common exam theme.

Practice Questions

Q1. Classify the following as Input, Output, or Both: Scanner, Monitor, Touchscreen, Speaker.

Solution: Scanner — Input; Monitor — Output; Touchscreen — Both (input via touch, output via display); Speaker — Output.

Q2. Differentiate Impact and Non-Impact printers with one example each.

Solution: Impact printers physically strike a ribbon against paper to form characters (e.g., Dot Matrix printer) — noisy, low cost, can print carbon copies. Non-Impact printers form characters without physical striking, using ink spray or toner (e.g., Laser printer) — quiet, higher quality, faster.

Q3. Why is a Plotter used instead of a regular printer for engineering drawings?

Solution: A plotter draws continuous lines using pens or ink jets over large sheets, giving high precision for vector graphics such as maps, architectural, and engineering drawings — something regular printers (optimized for text/raster images) handle poorly at large scale.

Previous Year Questions

1. Which of the following is purely an output device?\n(A) Scanner \n(B) Joystick \n(C) Plotter \n(D) Microphone

Answer: (C)

2. Dot Matrix printers belong to which category?\n(A) Non-impact \n(B) Impact \n(C) Thermal \n(D) Optical

Answer: (B)

3. A touchscreen is best classified as:\n(A) Input only \n(B) Output only \n(C) Both input and output \n(D) Storage device

Answer: (C)

1.6 Computer Software

Concept

Software is the set of instructions/programs that direct hardware to perform tasks. Broadly classified as: System Software — manages hardware and provides a platform for other software (e.g., Operating Systems like Windows/Linux, Device Drivers, Utility software like antivirus/disk cleanup, Language Translators — Compiler, Interpreter, Assembler); and Application Software — designed for end users to perform specific tasks (e.g., MS Word, browsers, tally, custom business apps), further divided into General-purpose (word processors, spreadsheets) and Specific-purpose (payroll systems). A Compiler translates an entire high-level program into machine code at once and reports all errors together; an Interpreter translates and executes line-by-line, stopping at the first error; an Assembler converts assembly language to machine code. Firmware is software permanently stored in ROM/hardware (e.g., BIOS) that manages basic device operations.

Practice Questions

Q1. Classify: Windows OS, MS Excel, Antivirus software, Device driver.

Solution: Windows OS — System software (Operating System); MS Excel — Application software; Antivirus — System software (Utility); Device driver — System software.

Q2. Differentiate a Compiler from an Interpreter.

Solution: Compiler: translates the whole program at once into machine code, produces an object file, faster execution afterward, all errors shown together (e.g., C compiler). Interpreter: translates and executes line-by-line, no separate object file, slower execution, stops at the first error encountered (e.g., Python interpreter).

Q3. What is Firmware? Give an example.

Solution: Firmware is low-level software permanently embedded in hardware (usually in ROM) that controls basic device functions and initializes hardware at startup. Example: BIOS/UEFI in a computer motherboard, or firmware in a printer/router.

Previous Year Questions

1. Which of these is System Software?
(A) MS PowerPoint (B) Operating System (C) Tally (D) Adobe Photoshop

Answer: (B)

2. A program that converts an entire high-level program into machine code before execution is a:
(A) Interpreter (B) Compiler (C) Linker (D) Loader

Answer: (B)

3. Antivirus software is an example of:
(A) Application software (B) Utility software (C) Firmware (D) Assembler

Answer: (B)

4. Software permanently stored in ROM to control basic hardware startup is called:
(A) Middleware (B) Firmware (C) Freeware (D) Shareware

Answer: (B)

1.7 The Software Problem (Software Crisis & Engineering)

Concept

The 'Software Problem' (also called the Software Crisis) refers to difficulties faced in developing reliable, on-time, on-budget software as systems grew complex in the 1960s-70s: projects running over budget/schedule, low quality, software that was hard to maintain, and mismatched user requirements. This led to the discipline of Software Engineering — applying systematic, disciplined, and measurable approaches to software development. Common symptoms of the software problem: poor requirement gathering, lack of proper planning/documentation, inadequate testing, and scope creep. Software Development Life Cycle (SDLC) models were introduced as solutions: Waterfall (sequential phases), Iterative/Incremental, Spiral (risk-driven), and Agile (flexible, customer-collaboration driven, works in short sprints). Understanding why software fails and how process models mitigate it is a recurring CPGET theme.

Practice Questions

Q1. List three common symptoms of the 'software crisis'.

Solution: 1) Projects exceeding budget and schedule. 2) Poor quality/unreliable software with many bugs. 3) Software not meeting actual user requirements (any three of: hard to maintain, poor documentation, scope creep are also valid).

Q2. Name the SDLC model that is sequential with no going back to a previous phase, and one drawback of it.

Solution: Waterfall model. Drawback: it does not allow easy revisiting of earlier phases, so requirement changes discovered late in the project are costly/difficult to incorporate.

Q3. How does the Agile model try to solve problems of the traditional Waterfall model?

Solution: Agile breaks development into small iterations/sprints with continuous customer feedback and flexibility to change requirements, unlike Waterfall's rigid, sequential, all-upfront-requirements approach — this reduces the risk of building the wrong product.

Previous Year Questions

1. The term describing widespread problems (cost/time overruns, poor quality) in large software projects is called the:
(A) Software Crisis (B) Hardware Crisis (C) Data Crisis (D) Network Crisis

Answer: (A)

2. Which SDLC model follows strict sequential phases with no overlap?
(A) Agile (B) Spiral (C) Waterfall (D) Prototype

Answer: (C)

3. The Spiral model is primarily driven by:
(A) Cost estimation (B) Risk analysis (C) Documentation (D) Testing only

Answer: (B)

1.8 Programming Principles and Guidelines

Concept

Good programming follows core principles: Modularity (breaking a program into smaller, manageable functions/modules), Readability (meaningful variable names, consistent indentation, comments), Reusability (writing generic code/functions usable in multiple contexts), and Maintainability (easy to update/debug later). Guidelines include: following a consistent naming convention, avoiding hard-coded 'magic numbers', proper error handling, minimizing global variables, writing DRY code (Don't Repeat Yourself), and thorough documentation. Algorithm design principles emphasize correctness (produces the right output), efficiency (time/space complexity), and simplicity. Structured programming principles (sequence, selection, iteration) avoid unrestricted 'goto' jumps, making logic easier to trace. Testing/debugging is an integral part of the coding process — unit testing checks individual modules, integration testing checks combined modules.

Practice Questions

Q1. What does the DRY principle in programming stand for, and why is it important?

Solution: DRY = 'Don't Repeat Yourself'. It means avoiding duplicate code by extracting common logic into reusable functions/modules, which reduces bugs and makes maintenance easier since a change needs to be made in only one place.

Q2. Name the three basic control structures of structured programming.

Solution: Sequence (statements executed one after another), Selection (decision-making, e.g., if-else), and Iteration (repetition, e.g., loops).

Q3. Why is meaningful variable naming considered good programming practice?

Solution: Meaningful names (e.g., 'totalMarks' instead of 'x') make code self-explanatory, improving readability and making it easier for the original author or others to understand, debug, and maintain the code later.

Previous Year Questions

1. Breaking a large program into smaller independent functions is called:\n(A) Modularity \n(B) Encapsulation \n(C) Polymorphism \n(D) Inheritance

Answer: (A)

2. Which of the following is NOT one of the three basic structured programming constructs?\n(A) Sequence \n(B) Selection \n(C) Iteration \n(D) Goto-jump

Answer: (D)

3. The principle of avoiding duplicate code by reusing logic is known as:\n(A) DRY \n(B) WET \n(C) KISS only \n(D) YAGNI only

Answer: (A)

4. Checking an individual function/module in isolation is called:\n(A) Integration testing \n(B) Unit testing \n(C) System testing \n(D) Acceptance testing

Answer: (B)

Revision Checklist

- Can distinguish Data vs Information with examples and know DIKW hierarchy
- Can convert numbers between binary, octal, decimal, and hexadecimal
- Know ASCII vs Unicode and why Unicode was introduced
- Can name and explain the 3 CPU components and the Fetch-Decode-Execute cycle
- Know LAN/MAN/WAN, topologies, OSI 7 layers, and network devices (hub/switch/router)
- Can classify devices as Input/Output/Both and printer types (Impact/Non-impact)
- Can classify software as System vs Application and explain Compiler vs Interpreter

- Understand the software crisis and SDLC models (Waterfall, Spiral, Agile)
- Know programming principles: modularity, DRY, structured programming constructs